

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 4 THE CORE OF OOP DAN MODUL 5
ENCAPSULATION



Disusun Oleh:

Yosafat Jacobus (105224016)

PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN KOMPUTER
UNIVERSITAS PERTAMINA

2026

DAFTAR ISI

I. Pendahuluan.....	2
II. Variabel.....	2
III. Constructor dan Method.....	4
IV. Dokumentasi dan Pembahasan Code.....	6
V. Kesimpulan.....	14

I. Pendahuluan

Disini saya membahas tentang implementasi konsep Enkapsulation dan Overloading, baik pada Constructor maupun Method. Disini saya membuat simulasi sistem manajemen kamar hotel sederhana beserta getter dan setter dilengkapi validasi, program ini memanfaatkan overloading untuk memberikan fleksibilitas dalam pembuatan objek kamar, proses pemesanan, dan perhitungan tagihan berdasarkan kondisi tertentu, seperti kapasitas tamu dan diskon promo.

II. Variabel

Disini ada class KamarHotel dan Main

No	Nama Variabel	Tipe Data	Fungsi
1	nomorKamar	String	Atribut untuk menyimpan nomor identitas kamar.
2	tipeKamar	String	Atribut untuk menyimpan kategori kamar (Reguler, Premium, Suite).
3	kapasitasMaksimal	int	Atribut untuk menyimpan batas maksimal tamu dalam satu kamar.
4	hargaPerMalam	double	Atribut untuk menyimpan harga sewa kamar per malam.

5	isTersedia	boolean	Atribut untuk menandai status ketersediaan kamar.
6	kamar1	KamarHotel	Objek instansiasi kamar pertama pada <i>class</i> Main.
7	kamar2	KamarHotel	Objek instansiasi kamar kedua pada <i>class</i> Main.
8	tagihanKamar1	double	Variabel lokal untuk menyimpan total harga sewa objek kamar1.
9	tagihanKamar2	double	Variabel lokal untuk menyimpan total harga sewa objek kamar2.
10	jumlahTamu	int	Parameter pada <i>method</i> pesanKamar untuk validasi kapasitas.
11	jumlahMalam	int	Parameter pada <i>method</i> hitungTotalBayar untuk mengalikan harga.
12	kodeVoucher	String	Parameter pada <i>method</i> hitungTotalBayar untuk mengecek promo diskon.

III. Constructor dan Method

No	Nama Metode	Jenis Metode	Fungsi
1	KamarHotel(String, String, int)	Constructor	Menginisialisasi objek kamar dengan nomor, tipe, dan kapasitas (harga <i>default</i> 0).
2	KamarHotel(String, String, int, double)	Constructor (Overloaded)	Menginisialisasi objek kamar lengkap beserta harga per malamnya.
3	getNomorKamar(), dll	Accessor (Getter)	Mengembalikan nilai dari atribut kamar yang bersifat <i>private</i> .
4	setTipeKamar(String)	Mutator (Setter)	Menetapkan tipe kamar dengan validasi hanya untuk tipe Reguler, Premium, dan Suite.
5	setHargaPerMalam(double)	Mutator (Setter)	Menetapkan harga per malam dengan validasi nominal minimal Rp50.000.

6	pesanKamar()	Method	Memproses pesanan kamar secara umum dan mengubah status ketersediaan.
7	pesanKamar(int)	Method (Overloaded)	Memproses pesanan dengan memvalidasi jumlah tamu agar tidak melebihi kapasitas.
8	batalPesan()	Method	Membatalkan pesanan dan mengembalikan status isTersedia menjadi true.
9	hitungTotalBayar(int)	Method	Menghitung total tagihan normal (harga * jumlah malam).
10	hitungTotalBayar(int, String)	Method (Overloaded)	Menghitung total tagihan dengan validasi voucher "PROMO" (diskon 20% jika menginap $\geq$ 3 malam).
11	cetakProfil()	Method	Mencetak detail informasi kamar ke layar konsol.
12	main(String[])	Method	Titik awal eksekusi program untuk menjalankan skenario pengujian hotel.

IV. Dokumentasi dan Pembahasan Code

Class KamarHotel

```
public class KamarHotel {  
    // Atribut Kamar  
    private String nomorKamar;  
    private String tipeKamar;  
    private int kapasitasMaksimal;  
    private double hargaPerMalam;  
    private boolean isTersedia;  
  
    //Membuat Konstruktor KamarHotel  
    public KamarHotel(String nomorKamar, String tipeKamar, int  
kapasitasMaksimal) {  
        this.nomorKamar = nomorKamar;  
        setTipeKamar(tipeKamar);  
        this.kapasitasMaksimal = kapasitasMaksimal;  
        this.hargaPerMalam = 0;  
        this.isTersedia = true;  
    }  
  
    public KamarHotel(String nomorKamar, String tipeKamar, int  
kapasitasMaksimal, double hargaPerMalam) {  
        this.nomorKamar = nomorKamar;  
        setTipeKamar(tipeKamar);  
        this.kapasitasMaksimal = kapasitasMaksimal;  
        setHargaPerMalam(hargaPerMalam);  
        this.isTersedia = true;  
    }  
}
```

1. Menggunakan Enkapsulasi

- Disini saya menggunakan private di atribut kamar (nomorKamar, tipeKamar, kapasitasMaksimal, hargaPerMalam, dan isTersedia) Ini adalah ini adalah konsep enkapsulasi, di mana data atau status dari objek disembunyikan dari akses kelas luar, supaya untuk menjaga integritas data agar tidak diubah sembarangan tanpa validasi

2. Penggunaan Constructor Overloading

- Constructor Pertama (3 Parameter): di buat objek KamarHotel baru dengan informasi dasar seperti nomorKamar, tipeKamar, kapasitasMaksimal. Di constructor ini, nilai hargaPerMalam diatur ke nilai default yaitu 0
- Constructor Kedua (4 Parameter): Memberikan fleksibilitas lebih bagi pengguna sistem untuk menginisialisasi objek KamarHotel lengkap hargaPerMalam-nya sejak awal objek dibuat

3. Penggunaan Keyword this

- Digunakan untuk membedakan anatara atribut milik class dengan parameter yang kebetulan mempunyai nama

4. Validasi Otomatis

- Constructor tidak mengisi atribut, melainkan memanggil setter seperti setTypeKamar dan setHargaPerMalam

```
//Membuat Getter untuk atribut kamar
public String getNomorKamar() { return nomorKamar; }
public String getTypeKamar() { return tipeKamar; }
public int getKapasitasMaksimal() { return kapasitasMaksimal; }
public double getHargaPerMalam() { return hargaPerMalam; }
public boolean getIsTersedia() { return isTersedia; }

public void setNomorKamar(String nomorKamar) {
    this.nomorKamar = nomorKamar;
}

public void setKapasitasMaksimal(int kapasitasMaksimal) {
    this.kapasitasMaksimal = kapasitasMaksimal;
}

//Membuat Setter untuk atribut kamar dengan validasi
public void setTypeKamar(String tipeKamar) {

    if (tipeKamar.equalsIgnoreCase("Reguler") ||
        tipeKamar.equalsIgnoreCase("Premium") ||
```



```

        tipeKamar.equalsIgnoreCase("Suite")) {

            // Jika tipe kamar valid, set tipe kamar
            if(tipeKamar.equalsIgnoreCase("Reguler"))
this.tipeKamar = "Reguler";
            else if(tipeKamar.equalsIgnoreCase("Premium"))
this.tipeKamar = "Premium";
            else this.tipeKamar = "Suite";
        } else {

            System.out.println("Peringatan: Tipe kamar '" +
tipeKamar + "' tidak dikenali. Sistem memaksanya menjadi
'Reguler'.");
            this.tipeKamar = "Reguler";
        }
    }

    public void setHargaPerMalam(double harga) {
        //Memvalidasi harga per malam, tidak boleh di bawah 50000
        if (harga < 50000) {
            System.out.println("Peringatan: Harga tidak boleh di
bawah 50000. Sistem memaksanya menjadi 50000.");
            this.hargaPerMalam = 50000;
        } else {
            this.hargaPerMalam = harga;
        }
    }
}

```

1. Metode Getter (Accessor): fungsi get ... () digunakan sebagai satu-satu jalan untuk membaca nilai dari atribut private seperti nomorKamar atau hargaPerMalam dari luar class.
2. Metode Setter (Mutator): Fungsi set ... () digunakan untuk memodifikasi/memperbarui nilai atribut
3. Validasi Tipe Data (setTipeKamar): setter ini mengamankan data dengan mengizinkan input Reguler, Premium, atau Suite menggunakan equalsIgnoreCase agar aman/kebal dari kesalahan huruf besar/kecil). Jika input asing dimasukkan, sistem otomatis akan memaksanya menjadi Reguler agar menghindari error pada perhitungan kategori
4. Validasi Logika Bisnis (setHargaPerMalam): Menggunakan statement

if-else untuk mencegah penetapan harga yang tidak logis. Jika input kurang dari Rp50.000 (termasuk angka minus juga), sistem menolaknya dan menetapkan harga ke batas minimum dasar Rp50.000

```
public void pesanKamar() {
    if (isTersedia) {
        isTersedia = false;
        System.out.println("Sukses: Kamar " + nomorKamar + "
berhasil dipesan.");
    } else {
        System.out.println("Peringatan: Kamar " + nomorKamar +
" sudah terisi!");
    }
}

// Overloading metode pesanKamar untuk jumlah tamu
public void pesanKamar(int jumlahTamu) {
    if (!isTersedia) {
        System.out.println("Peringatan: Kamar " + nomorKamar +
" sudah terisi!");
        return;
    }

    if (jumlahTamu > kapasitasMaksimal) {

        System.out.println("Teguran: Pemesanan ditolak! Jumlah
tamu (" + jumlahTamu + ") melebihi kapasitas maksimal (" +
kapasitasMaksimal + ").");

    } else {
        isTersedia = false;
        System.out.println("Sukses: Kamar " + nomorKamar + "
berhasil dipesan untuk " + jumlahTamu + " orang.");
    }
}
```

```

public void batalPesan() {
    isTersedia = true;
    System.out.println("Pemesanan dibatalkan. Kamar " +
    nomorKamar + " kini tersedia kembali.");
}

public double hitungTotalBayar(int jumlahMalam) {
    return hargaPerMalam * jumlahMalam;
}

//Membuat Overloading metode hitungTotalBayar untuk menghitung
dengan kode voucher
public double hitungTotalBayar(int jumlahMalam, String
kodeVoucher) {
    double total = hitungTotalBayar(jumlahMalam);

    if (kodeVoucher.equals("PROMO") && jumlahMalam >= 3) {
        double diskon = total * 0.20;
        return total - diskon;
    } else {

        System.out.println("Penolakan: Voucher tidak valid atau
syarat minimum menginap (3 malam) tidak terpenuhi. Tagihan normal
berlaku.");
        return total;
    }
}
}

```

1. Method Overloading (Polimorfisme): Diterapkan di fungsi pesanKamar dan hitungTotalBayar. Konsep ini menggunakan satu nama fungsi untuk skenario berbeda, dengan membedakan jumlah/tipe parameter yang dimasukkan.
2. Manajemen Status (pesanKamar dan batalPesan): Ini menggunakan variabel boolean isTersedia sebagai flag/penanda untuk mencegah double booking, dan mengembalikan status kamar menjadi tersedia saat pesanan dibatalkan.
3. Validasi Ekstra: pada pesanKamar(int jumlahTamu), fungsi ini dibuat

cerdas dengan mengecek dua kondisi sekaligus: apakah kamar kosong, dan apakah rombongan tidak melebihi kapasitasMaksimal.

4. Code Reusability: pada hitungTotalBayar variasi promo, sistem tidak menghitung manual (harga x malam) dari awal, ia malah memanggil variasi fungsi pertamanya yaitu hitungTotalBayat (jumlahMalam) untuk mendapatkan harga dasar, agar membuat kode lebih bersih dan menghindari perulangan.
5. Logika Bisnis Promosi: penggunaan operator logika AND (&&) untuk memastikan pemotongan diskon 20% sangat ketat; hanya berlaku jika input kode sama persis dengan “PROMO” dan durasi inap lebih dari atau sama dengan 3 malam.

```
//Membuat method untuk mencetak profil kamar
public void cetakProfil() {
    System.out.println("-----");
    System.out.println("Profil Kamar: " + nomorKamar);
    System.out.println("Tipe      : " + tipeKamar);
    System.out.println("Kapasitas  : " + kapasitasMaksimal + "
Orang");
    System.out.println("Harga/Malam : Rp " + hargaPerMalam);
    System.out.println("Tersedia    : " + (isTersedia ? "Ya" :
"Tidak"));
    System.out.println("-----");
}
}
```

Terakhir untuk Menampilkan Informasi Objek

- Fungsi cetakProfil untuk mencetak seluruh detail kamar (status atribut) ke layar konsol dengan format visual rapi dan terstruktur.
- Operator Ternary: Pada baris isTersedia ? “Ya” : “Tidak”), kode ini menggunakan operator logika ternary. Ini adalah cara efisien untuk menerjemahkan tipe data boolean (true/false) menjadi teks yang ramah pengguna (“Ya” atau “Tidak”) tanpa perlu menulis blok if-else yang panjang.
- String Concatenation: Program memanfaatkan operator + untuk merangkai teks statis (seperti “Harga/Malam: Rp “) dengan data dinamis yang diambil langsung dari atribut kelas.

Class Main

```
public class Main {  
    public static void main(String[] args) {  
  
        System.out.println("\n=== SIMULASI SISTEM HOTEL ===\n");  
        System.out.println("--- Pembuatan Kamar 1 ---");  
        KamarHotel kamar1 = new KamarHotel("101", "Reguler", 2);  
  
        kamar1.setTipeKamar("Presidential");  
        kamar1.setHargaPerMalam(-10000);  
        System.out.println();  
  
        System.out.println("--- Pembuatan Kamar 2 ---");  
        KamarHotel kamar2 = new KamarHotel("205", "Suite", 2,  
750000);  
  
        System.out.println("Mencoba memesan overcapacity:");  
        kamar2.pesanKamar(4);  
  
        System.out.println("\nMencoba memesan sesuai kapasitas:");  
        kamar2.pesanKamar(2);  
  
        System.out.println("\nMencoba memesan kamar yang sudah  
terisi:");  
        kamar2.pesanKamar();  
        System.out.println();  
  
        System.out.println("--- Perhitungan Tagihan ---");  
        System.out.println("Tagihan Kamar 1 (2 Malam, PROMO):");  
        double tagihanKamar1 = kamar1.hitungTotalBayar(2, "PROMO");  
        System.out.println("Total Bayar Kamar 1: Rp " +  
tagihanKamar1 + "\n");  
  
        System.out.println("Tagihan Kamar 2 (4 Malam, PROMO):");  
        double tagihanKamar2 = kamar2.hitungTotalBayar(4, "PROMO");  
        System.out.println("Total Bayar Kamar 2: Rp " +  
tagihanKamar2 + "\n");  
    }  
}
```

```

        System.out.println("=== STRUK INFORMASI AKHIR ===");
        kamar1.cetakProfil();
        kamar2.cetakProfil();
    }
}

```

1. Pengujian Enkapsulasi & Validasi Data (kamar1): Objek diuji dengan memberikan input yang salah secara sengaja, yaitu tipe kamar “Presidential” dan harganya -10000, sehingga program ini terbukti berhasil dan menolak input serta mengubah menjadi nilai default (“Reguler” dan Rp50.0000) berkat validasi di dalam setter.
2. Pengujian Fleksibilitas Constructor (kamar2): Menggunakan constructor 4 parameter untuk langsung membuat kamar "Suite" dengan harga spesifik (Rp750.000) dalam satu baris kode.
3. Pengujian Skenario Pesanan (Method Overloading sistem berhasil menangani tiga kondisi berbeda di kamar2:
 - Menolak saat pesanan tamu 4, sehingga ditolak, karena syaratnya melebihi kapasitas 2
 - Menerima pesanan saat kapasitas sesuai
 - Menolak pesanan baru saat status kamar sudah terisi (double-booking)
4. Pengujian Logika Promo: Menunjukkan perbandingan penerapan diskon. kamar1 ditolak promonya karena menginap 2 malam, karena syarat minimum 3 malam, sehingga tagihan normal tetap berlaku, Sedangkan kamar2 berhasil mendapatkan diskon 20% karena menginap selama 4 malam.
5. Verifikasi Akhir: Pemanggilan cetakProfil() di akhir program membuktikan bahwa semua status objek (seperti tipe kamar yang sudah dikoreksi dan status isTersedia yang menjadi “Tidak”) tersimpan dan diperbarui dengan benar.

V. Kesimpulan

Disimpulkan bahwa penerapan enkapsulasi(data hiding) sangat penting untuk menjaga keamanan dan integritas data. Penggunaan private yang dipadukan dengan metode getter dan setter memungkinkan program menyisipkan logika validasi, seperti batas minimum harga, dan filter tipe kamar sehingga memastikan bahwa sistem terlindungi dari manipulasi data atau input yang tidak logis yang dapat merusak jalannya

Selain itu, penerapan Overloading baik Constructor dan Method terbukti membuat program fleksibel dan dinamis, dan menggunakan satu nama metode yang sama, seperti pesanKamar atau hitungTotalBayar, untuk menangani skenario operasional yang berbeda hanya dengan membedakan jumlah atau tipe parameternya. Proses implementasi, penggunaan keyword this juga memegang peran penting atribut milik objek saat ini, agar mencegah terjadi ambiguitas atau bentrok nama. Struktur yang dibangun mematuhi prinsip dan code reusability atau dikenal dengan DRY (Don't Repeat Yourself), terlihat dari rancangan metode yang memanggil metode lain dalam kelas yang sama agar meminimalkan pengulangan penulisan kode tersebut.